

Question

Using the same data set (credit_card_data.txt or credit_card_data-headers.txt) as in Question 2.2, use the ksvm or kknns function to find a good classifier:

- (a) using cross-validation (do this for the k-nearest-neighbors model; SVM is optional); and
- (b) splitting the data into training, validation, and test data sets (pick either KNN or SVM; the other is optional).

Solution – (a):

Step 1: Load required library: kernlab supports SVM model, kknns supports KNN model.

```
#loading required libraries
library(kernlab)
library(kknn)
```

Step 2: Load data and checkout the dataset

```
#load data
data <- read.table("credit_card_data-headers.txt", header = TRUE)
#convert data to matrix
data_matrix <- as.matrix(data)

#prints dataset dimension
cat("Dimensions:", dim(data), "\n")

#prints the head data (top few rows)
print(head(data))

#prints summary of dataset
print(summary(data))
```

Step 3: Splitting the data into feature and target data

```
#let's split the data into feature and target (assume last column is target)
#feature data (it has to be converted to matrix form)
X <- as.matrix(data[, 1:(ncol(data)-1)])
#target data (it has to be converted to factor for classification)
y <- as.factor(data[,ncol(data)])
```

Step 4(a): Use the ksvm (linear) model to find accuracy

```
#using ksvm (linear)
model1 <- ksvm(X, y, type="C-svc", kernel="vanilladot", C=1)
pred1 <- predict(model1, X)
accuracy1 <- sum(pred1 == y) / length(y)
cat("KSVM Accuracy (Linear):", accuracy1, "\n")
```

OUTPUT

```
> #let's split the data into feature and target (assume last column is target)
> #feature data
> X <- as.matrix(data[, 1:(ncol(data)-1)])
> #target data (it has to be converted to factor for classification)
> y <- as.factor(data[,ncol(data)])
> model1 <- ksvm(X, y, type="C-svc", kernel="vanilladot", C=1)
Setting default kernel parameters
> pred1 <- predict(model1, X)
> accuracy1 <- sum(pred1 == y) / length(y)
> cat("KSVM Accuracy (Linear):", accuracy1, "\n")
KSVM Accuracy (Linear): 0.8639144
```

Step 4(b): Use the ksvm (RBF) model to find accuracy

```
#using ksvm (rbf)
model2 <- ksvm(X, y, type="C-svc", kernel="rbfdot", C=5)
pred2 <- predict(model2, X)
accuracy2 <- sum(pred2 == y) / length(y)
cat("KSVM Accuracy(RBF):", accuracy2, "\n")
```

OUTPUT

```
> model2 <- ksvm(X, y, type="C-svc", kernel="rbfdot", C=5)
> pred2 <- predict(model2, X)
> accuracy2 <- sum(pred2 == y) / length(y)
> cat("KSVM Accuracy(RBF):", accuracy2, "\n")
KSVM Accuracy(RBF): 0.9006116
```

Step 4(c): Use the 5-crossfold knn model to find accuracy

```
#using k-fold cross validation - KNN
set.seed(123)
k_values <- c(3, 5, 7, 9, 11, 15)
n_folds <- 5

#let's assume that k=3
best_k <- 3
best_acc <- 0

#try each k value to find the best one
for(k in k_values) {
  acc_sum <- 0

  for(fold in 1:n_folds) {
    # simple split
    test_size <- nrow(data) %% n_folds
    start_index <- (fold-1) * test_size + 1
    end_index <- min(fold * test_size, nrow(data))

    test_index <- start_index:end_index
    train_index <- setdiff(1:nrow(data), test_index)

    #create train and test data
    train_data <- data[train_index, ]
    test_data <- data[test_index, ]

    knn_model <- kknn(R1 ~ ., train_data, test_data, k = k)
    pred <- fitted(knn_model)
    acc <- mean(pred == test_data$R1)
    acc_sum <- acc_sum + acc
  }
  avg_acc <- acc_sum / n_folds
  cat("k =", k, "accuracy =", round(avg_acc, 3), "\n")

  if(avg_acc > best_acc) {
    best_acc <- avg_acc
    best_k <- k
  }
}
```

OUTPUT

```
k = 3 accuracy = 0.592
k = 5 accuracy = 0.5
k = 7 accuracy = 0.414
k = 9 accuracy = 0.343
k = 11 accuracy = 0.286
k = 15 accuracy = 0.168
>
> cat("Best k =", best_k, "with accuracy =", round(best_acc, 3), "\n")
Best k = 3 with accuracy = 0.592
```

Step 5: Finding the best classifier – based on accuracy

```
cat("Best Accuracy & Model: ", "\n")
|
if((accuracy1>=accuracy2) && (accuracy1>=accuracy3)){
  cat("KSVM (Linear) - Accuracy: ", accuracy1, "\n")
}else if(accuracy2>=accuracy3){
  cat("KSVM (RBF) - Accuracy: ", accuracy2, "\n")
}else{
  cat("KNN Crossfold - Accuracy: ", accuracy3, "\n")
}
```

OUTPUT:

```
Best Accuracy & Model:
>
> if((accuracy1>=accuracy2) && (accuracy1>=accuracy3)){
+   cat("KSVM (Linear) - Accuracy: ", accuracy1, "\n")
+ }else if(accuracy2>=accuracy3){
+   cat("KSVM (RBF) - Accuracy: ", accuracy2, "\n")
+ }else{
+   cat("KNN Crossfold - Accuracy: ", accuracy3, "\n")
+ }
KSVM (RBF) - Accuracy: 0.9036697
```

Solution - (b): I am using KSVM RBF approach because it had high accuracy.

#3.2b - split into train and test data - using KSVM

```
set.seed(123)
n <- nrow(data)
train_size <- floor(0.7 * n)

sample_index <- sample(1:n)
train_index <- sample_index[1:train_size]
test_index <- sample_index[(train_size + 1):n]

train_data <- data[train_index, ]
test_data <- data[test_index, ]

cat("Train size:", nrow(train_data), "\n")
cat("Test size:", nrow(test_data), "\n")

#convert target to factor
train_data$R1 <- as.factor(train_data$R1)
test_data$R1 <- as.factor(test_data$R1)

#simple SVM model
svm_model <- ksvm(R1 ~ ., data = train_data, kernel = "rbfdot")

#predictions
train_pred <- predict(svm_model, train_data)
test_pred <- predict(svm_model, test_data)

#accuracy
train_acc <- mean(train_pred == train_data$R1)
test_acc <- mean(test_pred == test_data$R1)

cat("Train accuracy:", round(train_acc, 3), "\n")
cat("Test accuracy:", round(test_acc, 3), "\n")
```

OUTPUT

```
> cat("Train size:", nrow(train_data), "\n")
Train size: 457
> cat("Test size:", nrow(test_data), "\n")
Test size: 197
> cat("Train accuracy:", round(train_acc, 3), "\n")
Train accuracy: 0.877
> cat("Test accuracy:", round(test_acc, 3), "\n")
Test accuracy: 0.843
```

CONCLUSION:

The model with best accuracy is KSVM(RBF).